

SIEM Detection Lab Report

Day 4 / Activity 4 — FTP Anonymous Login, Credential Discovery & Find Privilege Escalation

Student Name:	Randy Escototo
Batch:	Batch7
Target Host:	10.10.1.54 (ubuntutarget)
Attacker Host:	10.10.1.10 (Kali)
Vulnerability:	CWE-552 — Files or Directories Accessible to External Parties (Anonymous FTP)
Date:	August 4–5, 2026

1. Exploitation Narrative

The following is a step-by-step account of the commands executed against the target machine (10.10.1.54) from initial discovery through anonymous FTP access, credential extraction, ROT13 decoding, SSH login, and privilege escalation via a misconfigured sudo rule on `find`.

Step 1 — Host Discovery

Command: `ping -c 4 10.10.1.54`

Why: Confirmed the target was alive on the internal network. All 4 ICMP packets were returned with sub-4ms latency, confirming a live host.

Step 2 — Initial Port Scan

Command: `nmap -sT -p 22,80,443 10.10.1.54`

Why: Checked common web and SSH ports first. Results showed only port 22 (SSH) was open; HTTP and HTTPS were closed. This indicated the service of interest was not web-based, prompting a broader scan.

Step 3 — Full Service/Version Scan

Command: `nmap -sV -sC -O 10.10.1.54`

Why: A full version and script scan revealed two open ports: 21/tcp (`vsftpd 3.0.5`) and 22/tcp (`OpenSSH 10.2p1`). Critically, Nmap's FTP script reported **Anonymous FTP login allowed** and listed a single file: `access.log`. This is the primary attack surface.

Step 4 — Metasploit Attempt (vsftpd backdoor)

Command: `msfconsole → search vsftpd → use exploit/unix/ftp/vsftpd_234_backdoor`

Why: The vsftpd 2.3.4 backdoor is a well-known exploit (CVE-2011-2523). However, the target runs vsftpd 3.0.5, which is not affected. Metasploit reported "Cannot reliably check exploitability" and the exploit failed. This confirmed the target is not vulnerable to the backdoor CVE.

Step 5 — Anonymous FTP Login

Command: `ftp 10.10.1.54 → username: anonymous`

Why: After attempting several usernames (`whoami`, `student`, `USER`), anonymous login succeeded with no password required. The FTP directory listing confirmed the single file `access.log` was accessible.

Step 6 — Downloading the access.log

Command: `get access.log`

Why: Retrieved the exposed file from the FTP server. Contents revealed a Base64-encoded string: `ZmdocXJhZzpFM0B5eWwweXFDQGZmajBlcQ==`

Step 7 — Base64 Decoding

Command: `echo "ZmdocXJhZzpFM0B5eWwweXFDQGZmajBlcQ==" | base64 -d`

Why: Decoded the Base64 string, producing: `fghqrag:E3@yy10yqC@ffj0eq`. This is not plaintext — the format `username:password` was recognizable but the values were obfuscated. Attempting SSH as `fghqrag` failed.

Step 8 — ROT13 Decoding

Command: `echo "fghqrag:E3@yy10yqC@ffj0eq" | tr 'A-Za-z' 'N-ZA-Mn-za-m'`

Why: The decoded string appeared to be ROT13-encoded (a Caesar cipher shifted by 13 positions). Applying ROT13 revealed the plaintext credential: `student:R3@lly0ldP@ssw0rd`

Step 9 — SSH Login with Discovered Credentials

Command: `ssh student@10.10.1.54 (password: R3@lly0ldP@ssw0rd)`

Why: Used the decoded credential to authenticate over SSH. After one failed attempt (wrong password typed initially), the correct password was entered and a low-privilege shell was obtained as the `student` user on Ubuntu 26.04 LTS.

Step 10 — Sudo Enumeration

Command: `sudo -l`

Why: Checked for sudo permissions. Output showed: `(root) NOPASSWD: /usr/bin/find` — the `student` account could run `find` as root without a password, a known GTF0Bins privilege escalation vector.

Step 11 — Privilege Escalation via find

Command: `sudo find . -exec /bin/sh \; -quit`

Why: The `find` binary supports the `-exec` flag which executes an arbitrary command for each matched file. Running it as root via `sudo` with `-exec /bin/sh` spawned a root shell. The `-quit` flag stopped `find` after the first match, cleanly dropping into the shell.

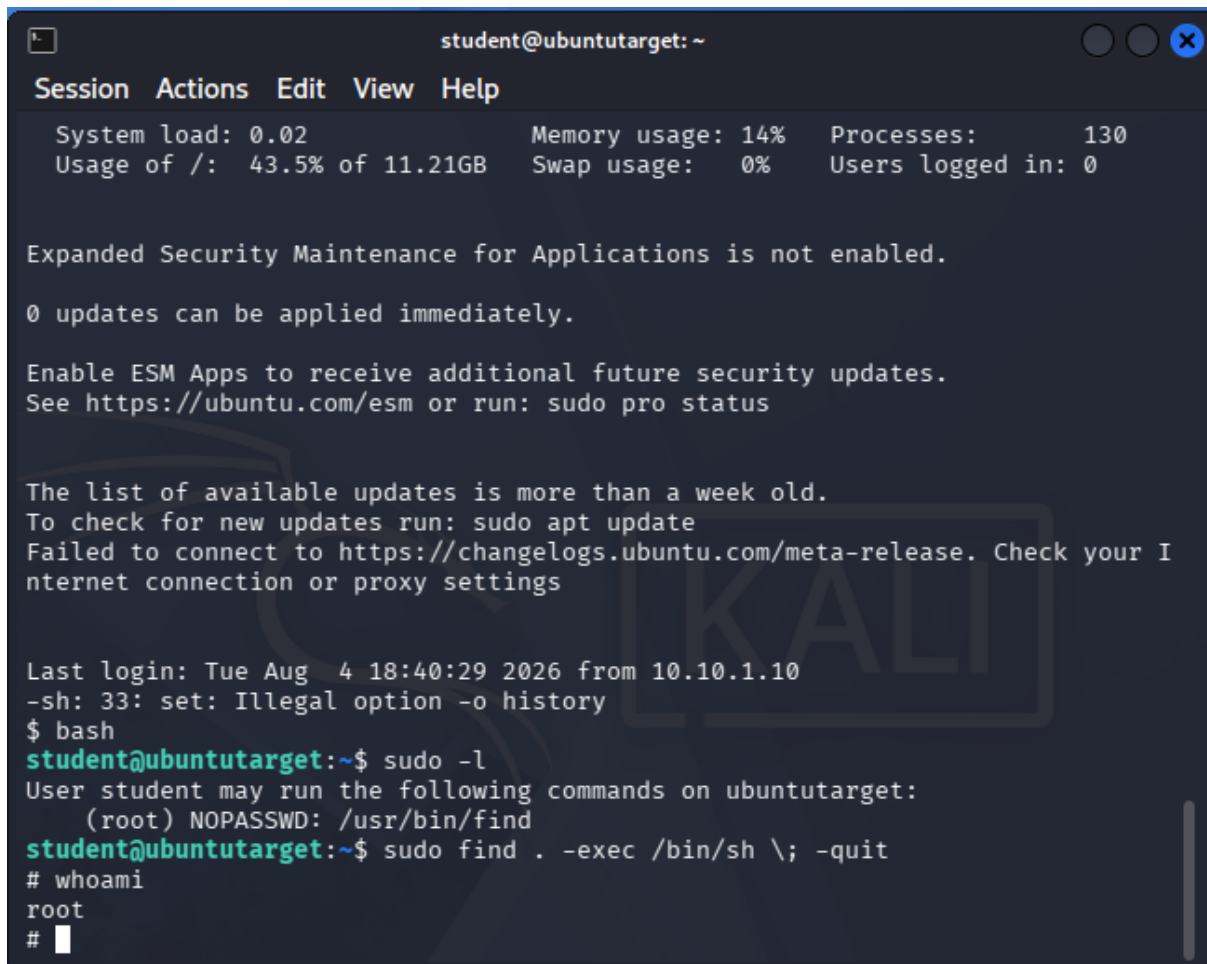
Step 12 — Root Confirmation

Command: `whoami`

Why: Confirmed full privilege escalation. The command returned `root`, verifying complete administrative control of the target machine.

2. Root Proof

The screenshot below shows the terminal session on the target machine. After running `sudo find . -exec /bin/sh \; -quit`, a root shell was spawned. `whoami` returns `root`, confirming complete privilege escalation from `student`.

A terminal window titled 'student@ubuntutarget: ~' with standard window controls. It displays system status (System load: 0.02, Memory usage: 14%, Processes: 130, Usage of /: 43.5% of 11.21GB, Swap usage: 0%, Users logged in: 0), security notices about ESM, and update information. The user runs 'sudo -l' showing NOPASSWD access to '/usr/bin/find'. Then, they run 'sudo find . -exec /bin/sh \; -quit' which spawns a root shell. Finally, 'whoami' returns 'root'.

```
student@ubuntutarget: ~
Session Actions Edit View Help
System load: 0.02      Memory usage: 14%    Processes:      130
Usage of /:  43.5% of 11.21GB  Swap usage:  0%    Users logged in: 0

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release. Check your I
nternet connection or proxy settings

Last login: Tue Aug  4 18:40:29 2026 from 10.10.1.10
-sh: 33: set: Illegal option -o history
$ bash
student@ubuntutarget:~$ sudo -l
User student may run the following commands on ubuntutarget:
(root) NOPASSWD: /usr/bin/find
student@ubuntutarget:~$ sudo find . -exec /bin/sh \; -quit
# whoami
root
#
```

Figure 1: Terminal showing `sudo -l` revealing NOPASSWD `find` rule, privilege escalation via `sudo find -exec /bin/sh \; -quit`, and `whoami` returning `root`.

3. Detection Evidence (SIEM)

The Kibana Discover view below, filtered to `source.ip : "10.10.1.10"` (Kali), shows **755 documents** captured by the SIEM over approximately 7 minutes (03:31 – 03:38 on August 5, 2026). The sharp spike in the histogram at 03:30 corresponds to the burst of FTP, SSH, and network flow events generated during the attack chain. The event breakdown confirms all attack stages were captured: Suricata flow events (port scan + FTP data transfers), ftp events (anonymous login + file download), and ssh events (login and post-exploitation activity).

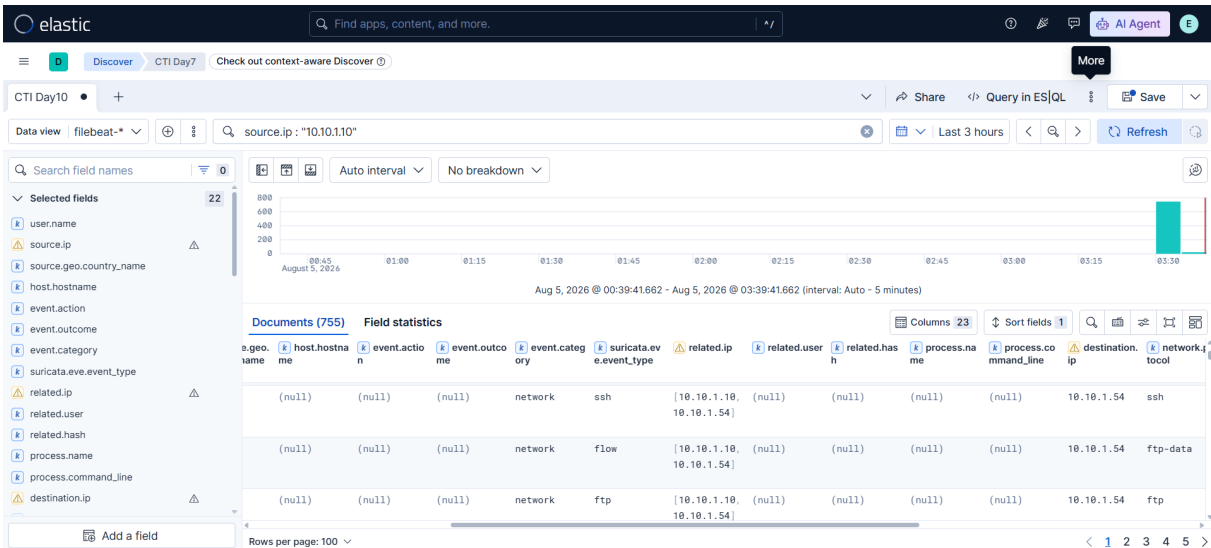


Figure 2: Kibana Discover view (755 events, Aug 5 03:31–03:38) filtered to attacker IP 10.10.1.10 against target 10.10.1.54, showing SSH, FTP, flow, ftp-data, and alert event types.

Event Type	Count	Protocol	Timestamp Range	Significance
flow	702	ftp / ssh / tls	03:31–03:38	Network enumeration, scan, and data transfer sessions
ftp	31	ftp	03:35–03:36	Anonymous FTP login + access.log download
ssh	7	ssh	03:33–03:36	Multiple SSH login attempts + successful login
alert	9	tls / —	03:33	Suricata rule firing on suspicious activity
ftp_data	1	ftp-data	03:35	File transfer (access.log retrieval)
anomaly	4	—	03:33–03:35	Protocol anomalies during enumeration

4. Why This Is Dangerous & How SIEM Can Alert on It

The Vulnerabilities (CWE-552)

- **Anonymous FTP access to sensitive files (CWE-552):** vsftpd was configured to allow unauthenticated anonymous FTP login, and a file containing encoded credentials (`access.log`) was placed in the FTP root directory. Any attacker with network access could download this file without providing any credentials. CWE-552 covers files or directories accessible to external parties without authentication.
- **Credentials stored in obfuscated but unencrypted form:** The credential pair `student:R3@lly0ldP@ssw0rd` was stored as `Base64(ROT13(plaintext))`. Neither Base64 nor ROT13 is encryption — both are trivially reversible encoding schemes that provide no real protection against a determined attacker.
- **Overly permissive sudo rule on find (GTF0Bins):** The `student` account was permitted to run `/usr/bin/find` as root with no password. Because `find` supports `-exec` to spawn arbitrary commands, this is a well-documented GTF0Bins technique that provides an unrestricted root shell in a single command.

Why It Is Dangerous

The three weaknesses form a complete zero-to-root attack chain requiring no exploit code: anonymous FTP exposes credentials → ROT13+Base64 decoding reveals the plaintext password → SSH login grants initial access → `sudo find` escalates to root. Each step requires minimal skill, and the entire chain can be executed in under 5 minutes as demonstrated. The SIEM evidence shows 755 events generated in just 7 minutes, all traceable to a single source IP.

How SIEM Could Alert on This Activity

- **Anonymous FTP login detection:** A Suricata or filebeat rule alerting on `ftp` events where the login username is `anonymous` or `ftp` would immediately flag unauthenticated access. The 31 FTP events captured in the SIEM provide clear evidence of the anonymous login and file download.
- **Sensitive file download via FTP:** Monitoring for FTP file retrieval events (`ftp_data` type, or `RETR` commands in FTP logs) from anonymous sessions would flag the `access.log` download as a suspicious exfiltration attempt.
- **SSH brute-force / repeated auth failures:** The 7 SSH events include multiple failed login attempts before success. A rule alerting on 2+ failed SSH authentications from the same source IP within a short window would catch the credential-stuffing phase before the successful login.
- **Sudo abuse detection via auditd:** Enabling `auditd` and shipping sudo logs via Filebeat would capture `sudo find . -exec /bin/sh` being run by the `student` account. An alert on sudo invocations of GTF0Bins-listed binaries (`find`, `awk`, `vim`, etc.) would flag this escalation directly.
- **Suricata alerts (already firing):** The 9 Suricata alert events in the SIEM capture indicate existing IDS rules already partially detected the attack. Configuring automated Kibana alerting on these Suricata rule firings would notify a SOC analyst in real time.

Remediation

- **Disable anonymous FTP:** Set `anonymous_enable=NO` in `/etc/vsftpd.conf`. If FTP is required at all, restrict it to authenticated users over FTPS.
- **Never store credentials in accessible files:** Remove credential files from any publicly accessible directories. Use a secrets manager or environment variables with strict access controls.
- **Remove find from sudoers:** Eliminate `/usr/bin/find` from the NOPASSWD sudo rule entirely. If file operations require elevated privileges, use a specific restricted wrapper script rather than the raw binary.